Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра инструментального и прикладного программного обеспечения

Дисциплина «Проектирование и разработка серверных частей интернет-ресурсов»

КУРСОВАЯ РАБОТА

Серверная часть веб-приложения «Расписание»

Студент: Васильев Б.А.
Группа: ИКБО-20-23
Руководитель: старший преподаватель Романченко А.Е.

Москва 2025

Цель

Проектирование и разработка серверной части веб-приложения «Расписание»

Задачи

1. Провести анализ предметной области разрабатываемого веб-приложения
2. Обосновать выбор технологий разработки веб-приложения
3. Разработать архитектуру веб-приложения на основе выбранного паттерна проектирования
4. Реализовать слой серверной логики веб-приложения с применением выбранной технологии
5. Реализовать слой логики базы данных
6. Разработать слой клиентского представления веб-приложения
7. Создать презентацию по выполненной курсовой работе

Анализ существующих аналогов

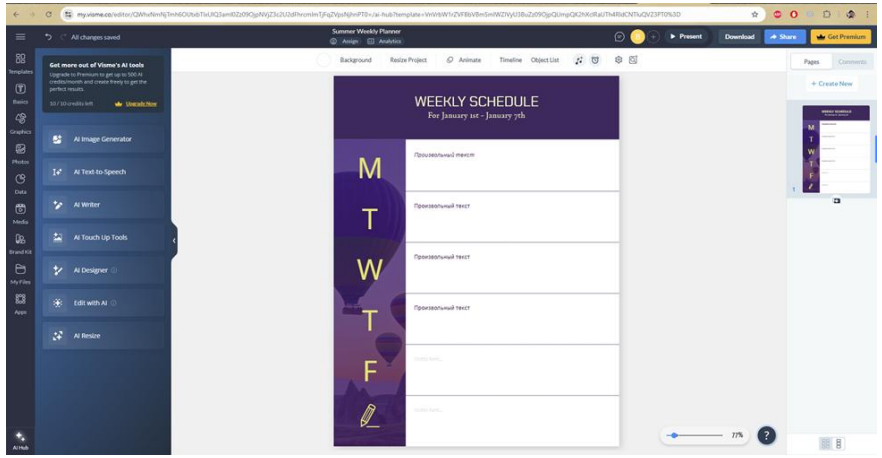

Рисунок 1 – Скриншот интерфейса сервиса Visme

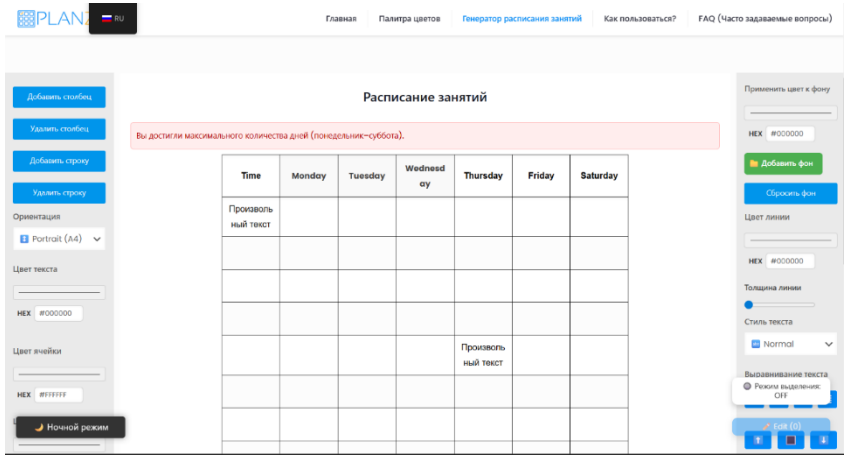

Рисунок 2 – Скриншот интерфейса сервиса Planzeo

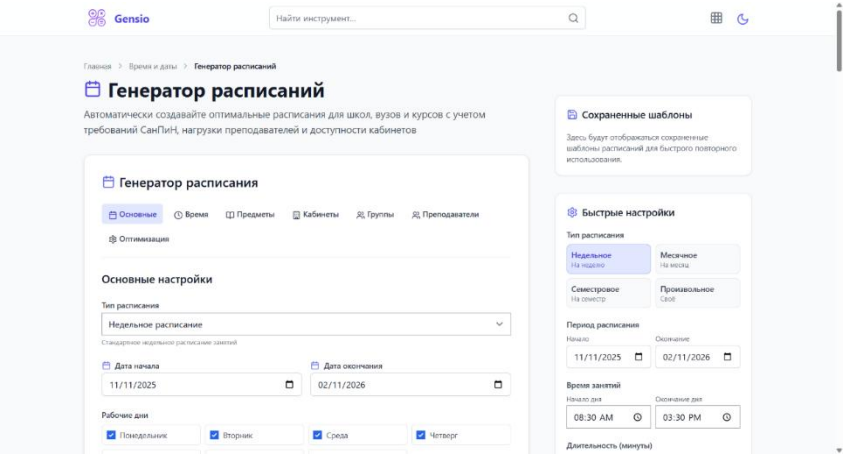

Рисунок 3 – Скриншот интерфейса сервиса Gensio

Определение требований к приложению

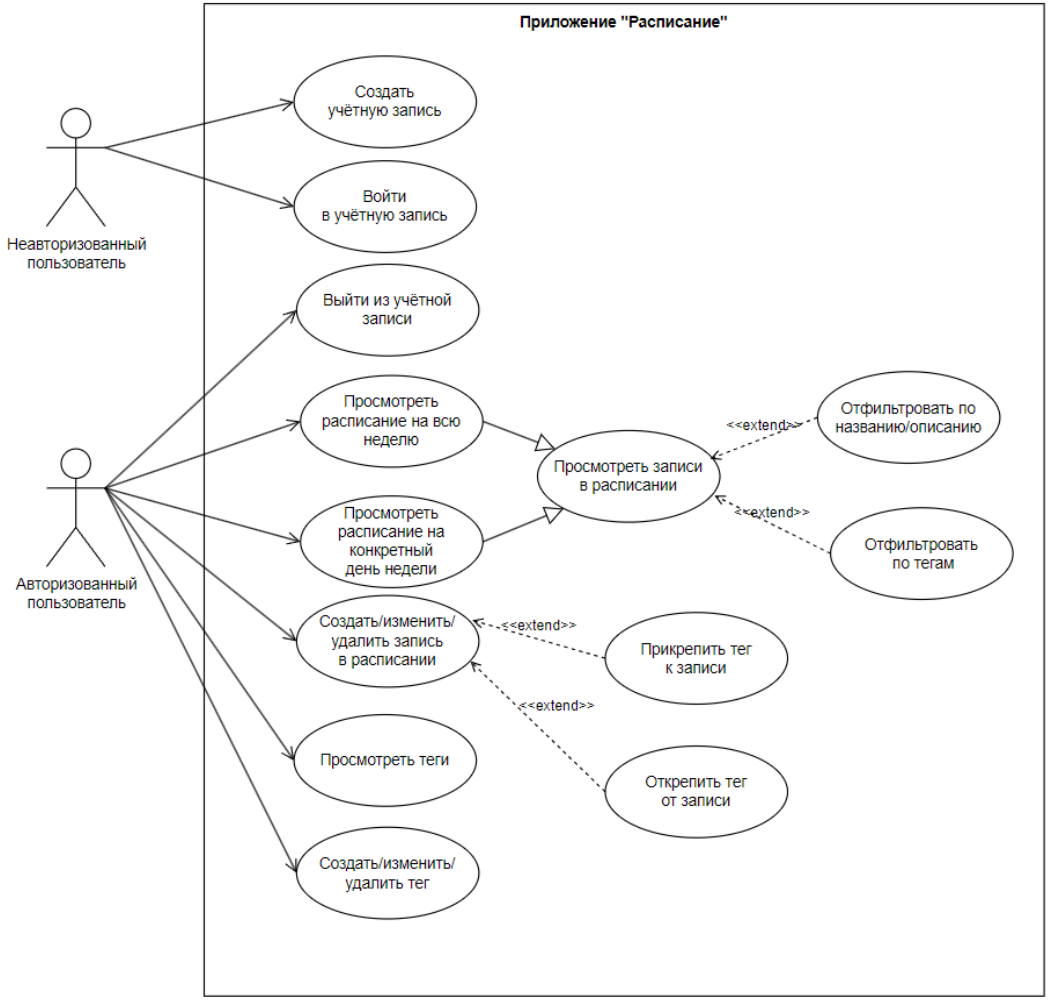

Рисунок 4 – UML-диаграмма вариантов использования приложения

Технологии разработки

HTML5

CSS3

JavaScript

PHP

Laravel

PostgreSQL

Docker

Visual Studio Code

Проектирование приложения

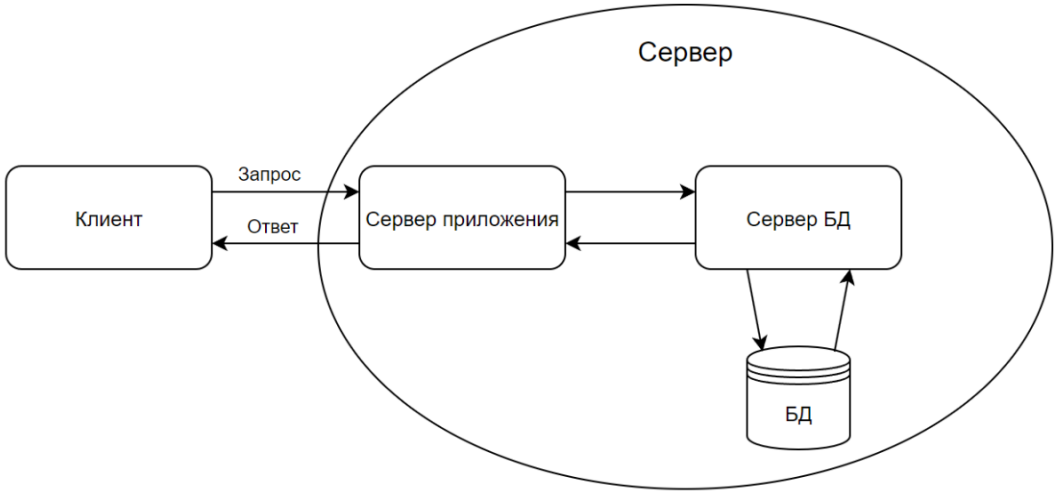

Рисунок 5 – Схема трёхзвенной клиент-серверной архитектуры

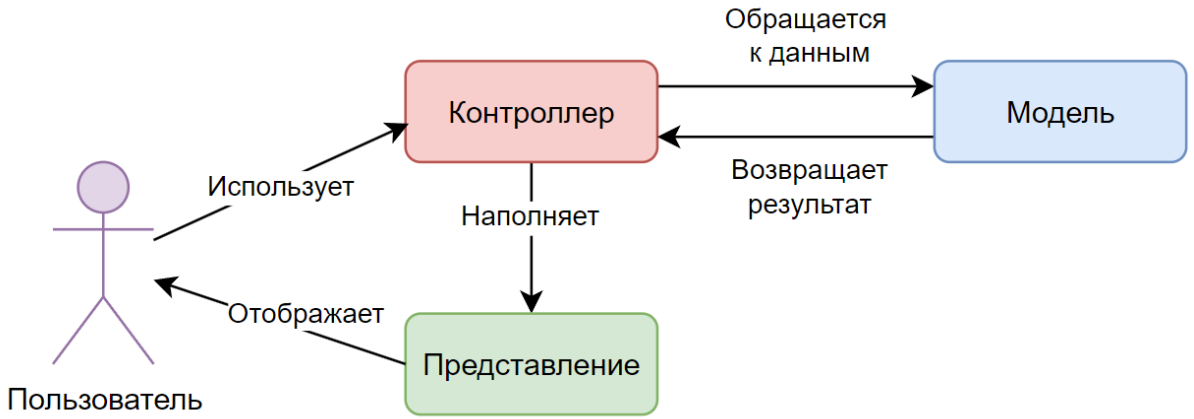

Рисунок 6 – Схема паттерна проектирования MVC

Проектирование приложения

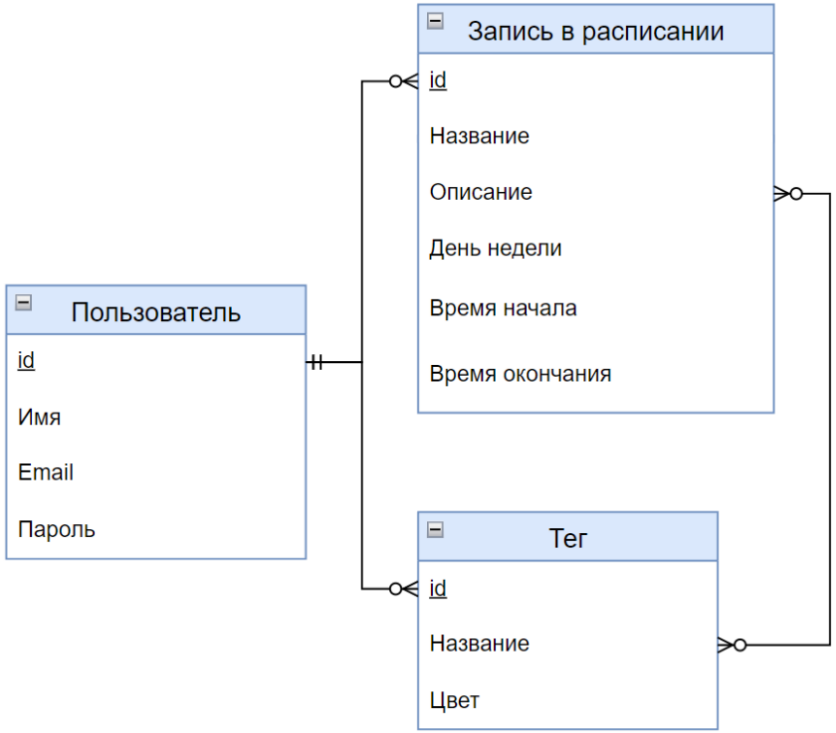

Рисунок 6 – ER-диаграмма базы данных

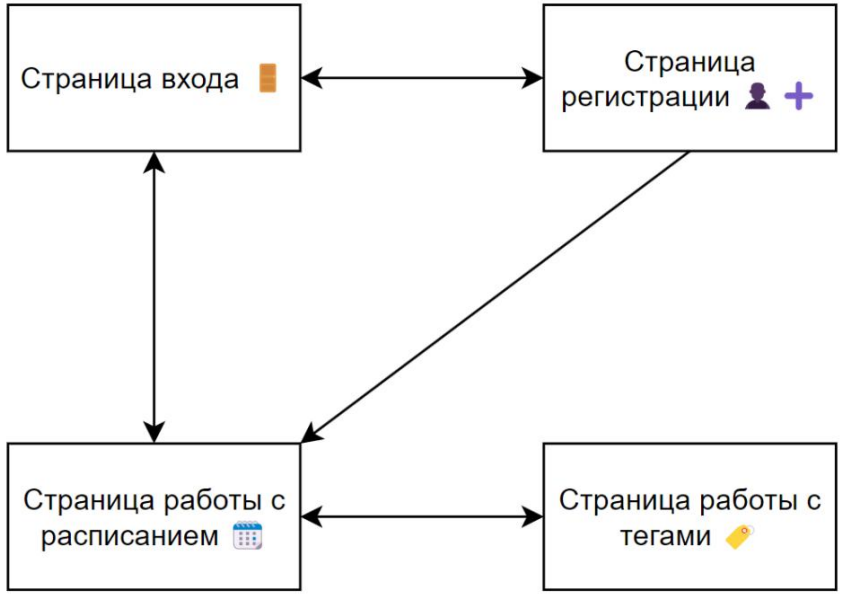

Рисунок 7 – Схема навигации между страницами веб-приложения

Реализация слоя данных: миграции

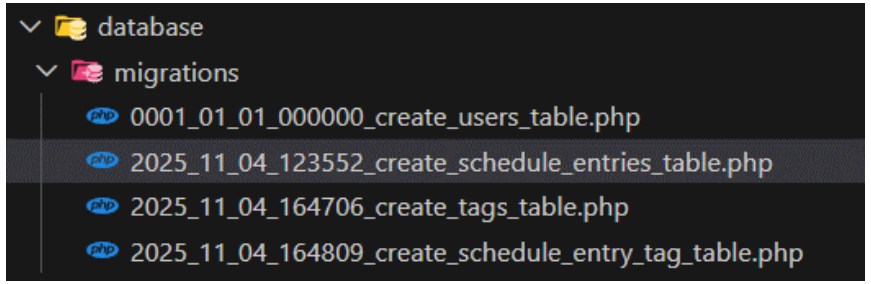

Рисунок 8 – Созданные миграции

```php
/**
 * Run the migrations.
 */
public function up(): void
{
    Schema::create('schedule_entries', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade'); // Внешний ключ к таблице users
        $table->string('title');
        $table->text('description')->nullable();
        $table->integer('day_of_week'); // 1 = Пн, 7 = Вс
        $table->time('start_time');
        $table->time('end_time');
        $table->timestamps();
    });

    // Добавляем ограничения (check constraint) после создания таблицы
    DB::statement('ALTER TABLE schedule_entries ADD CONSTRAINT check_end_time_gt_start_time CHECK (end_time > start_time)');
    DB::statement('ALTER TABLE schedule_entries ADD CONSTRAINT check_day_of_week_range CHECK (day_of_week BETWEEN 1 AND 7)');
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('schedule_entries');
}
```

Рисунок 9 – Фрагмент миграции по созданию таблицы для сущности «Запись в расписании»

Реализация слоя данных: модели

Рисунок 10 – Созданные модели

```php
class ScheduleEntry extends Model
{
    /**...
    protected $fillable = [
        'user_id',
        'title',
        'description',
        'day_of_week',
        'start_time',
        'end_time',
    ];

    /**...
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    /**...
    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

Рисунок 11 – Фрагмент модели сущности «Запись в расписании»

Реализация слоя серверной логики: маршрутизация

```php
// Главная страница перенаправляет на страницу расписания
Route::get('/', function () {
    return redirect('/schedule-entries');
});

// Маршруты для аутентификации
Route::group([], function() {
    // Маршруты входа
    Route::get('login', [LoginController::class, 'showLoginForm'])->name('login');
    Route::post('login', [LoginController::class, 'login']);

    // Маршрут выхода
    Route::post('logout', [LoginController::class, 'logout'])->name('logout');

    // Маршруты регистрации
    Route::get('register', [RegisterController::class, 'showRegistrationForm'])->name('register');
    Route::post('register', [RegisterController::class, 'register']);
});


// Группа маршрутов работы с бизнес-сущностями
// Требует аутентификации
Route::middleware(['auth'])->group(function () {
    // Маршруты для работы с ScheduleEntry
    Route::resource('schedule-entries', ScheduleEntryController::class)->only([
        'index', 'store', 'update', 'destroy'
    ]);

    // Маршруты для работы с Tag
    Route::resource('tags', TagController::class)->only([
        'index', 'store', 'update', 'destroy'
    ]);
});
```

Рисунок 12 – Фрагмент файла маршрутизации запросов

Реализация слоя серверной логики: контроллеры

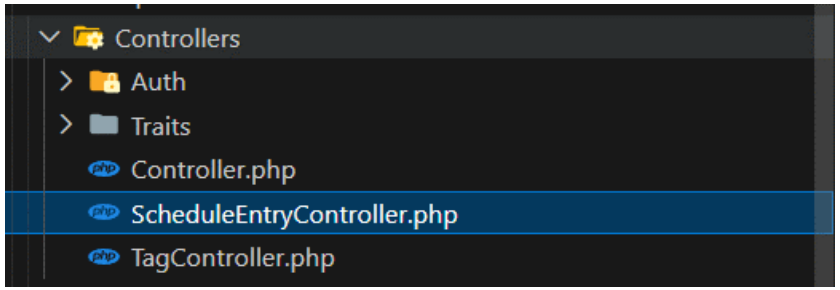

Рисунок 13 – Созданные контроллеры

```php
public function update(UpdateScheduleEntryRequest $request, ScheduleEntry $schedule_entry)
{
    $validated = $request->validated();
    $schedule_entry->update($validated);

    // Прикрепляем теги, если они были переданы
    if (isset($validated['tags'])) {
        $schedule_entry->tags()->sync($validated['tags']);
    } else {
        // Если теги не переданы (например, все были сняты), отсоединяем все теги
        $schedule_entry->tags()->detach();
    }

    // Формируем успешный ответ с URL для редиректа.
    $redirectUrl = $this->getBackUrl($this->fallbackRoute);
    return JsonResponseBuilder::success('Запись успешно обновлена.', ['redirectUrl' => $redirectUrl]);
}
```

Рисунок 14 – Фрагмент метода контроллера записей для обновления

Реализация слоя клиентского представления

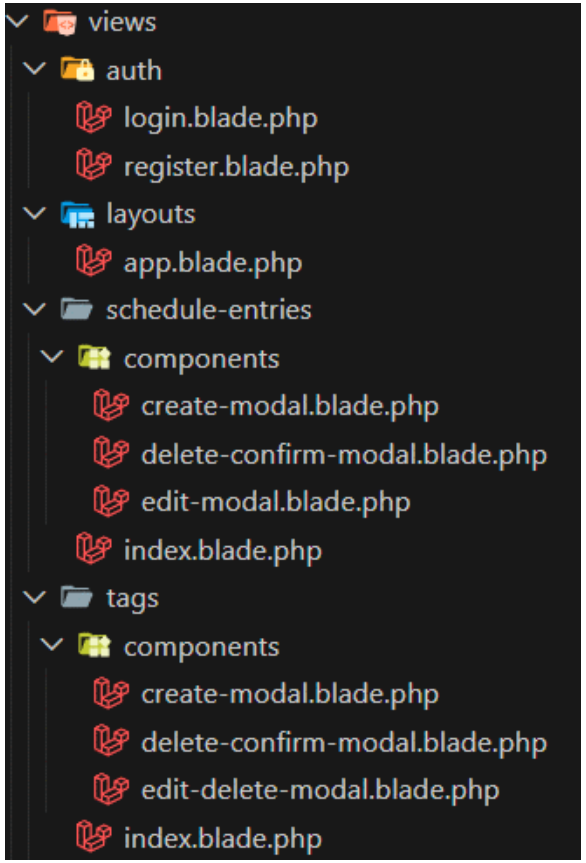

Рисунок 15 – Созданные шаблоны Blade

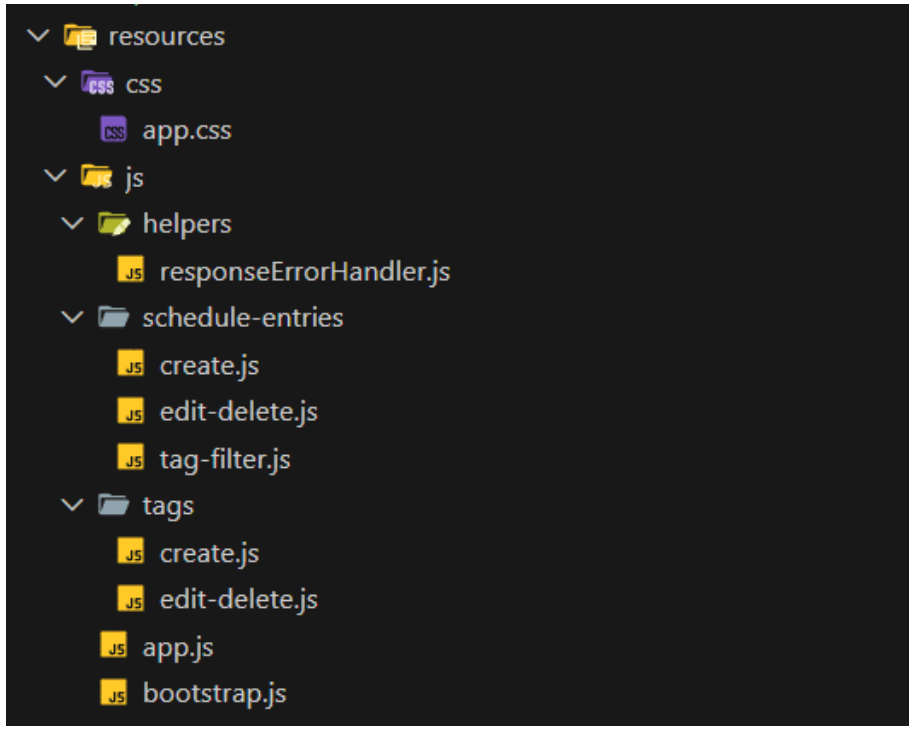

Рисунок 16 – Созданные файлы стилей и клиентской логики

Итоговый вид веб-приложения

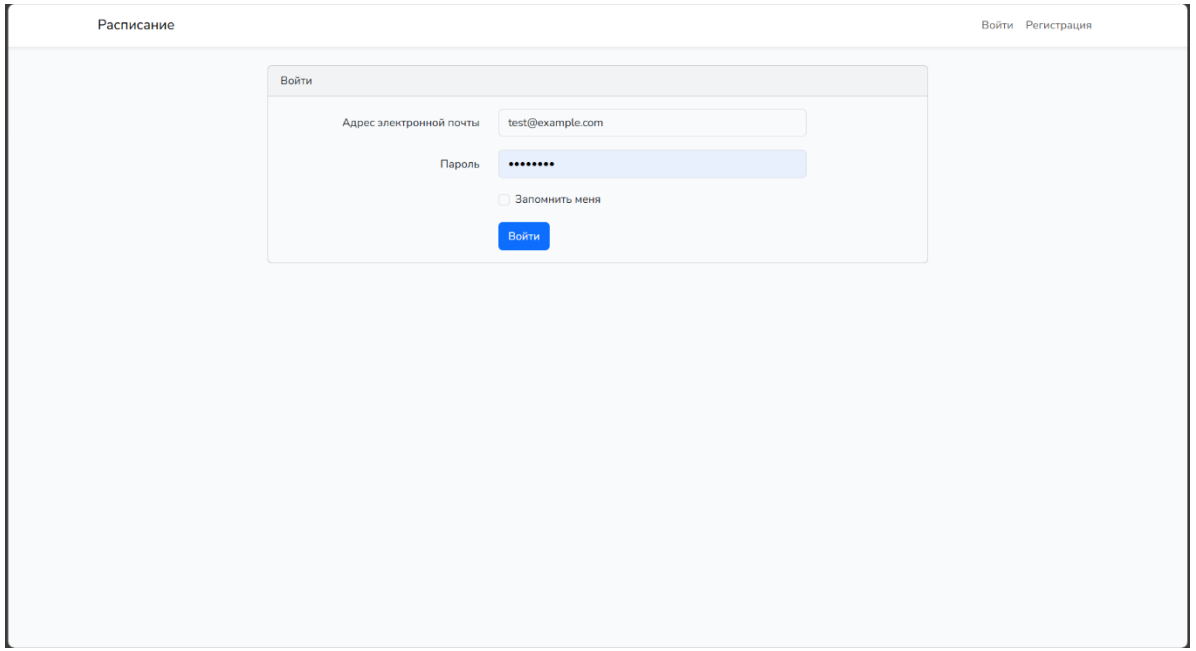

Рисунок 17 – Страница входа

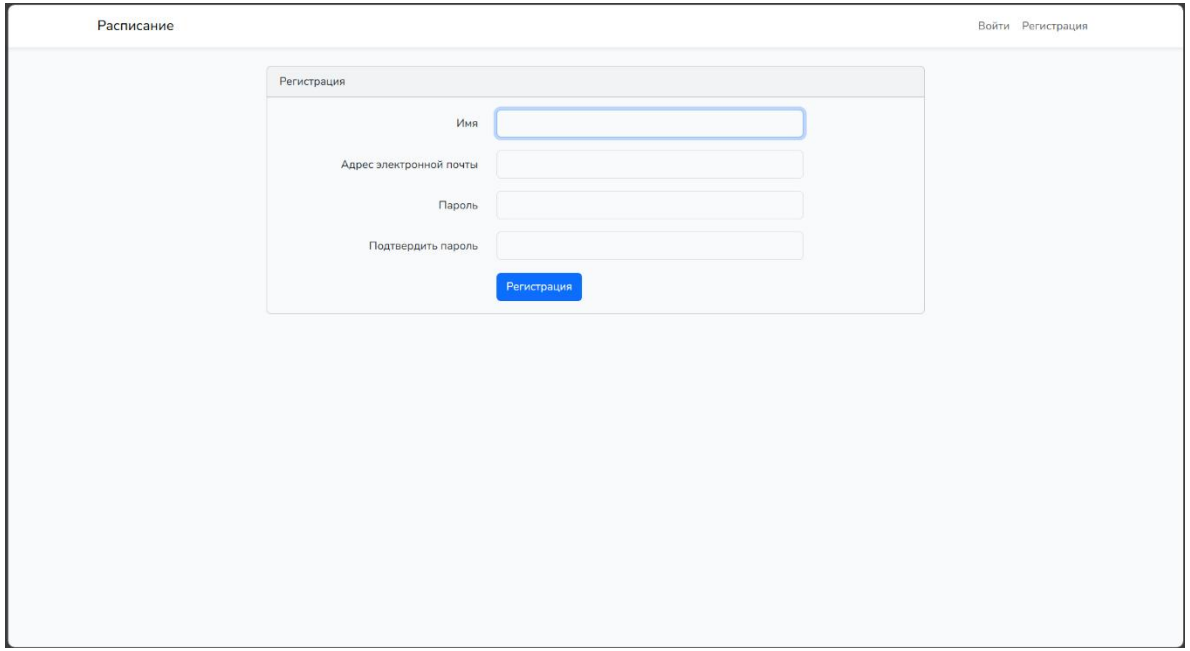

Рисунок 18 – Страница регистрации

Итоговый вид веб-приложения

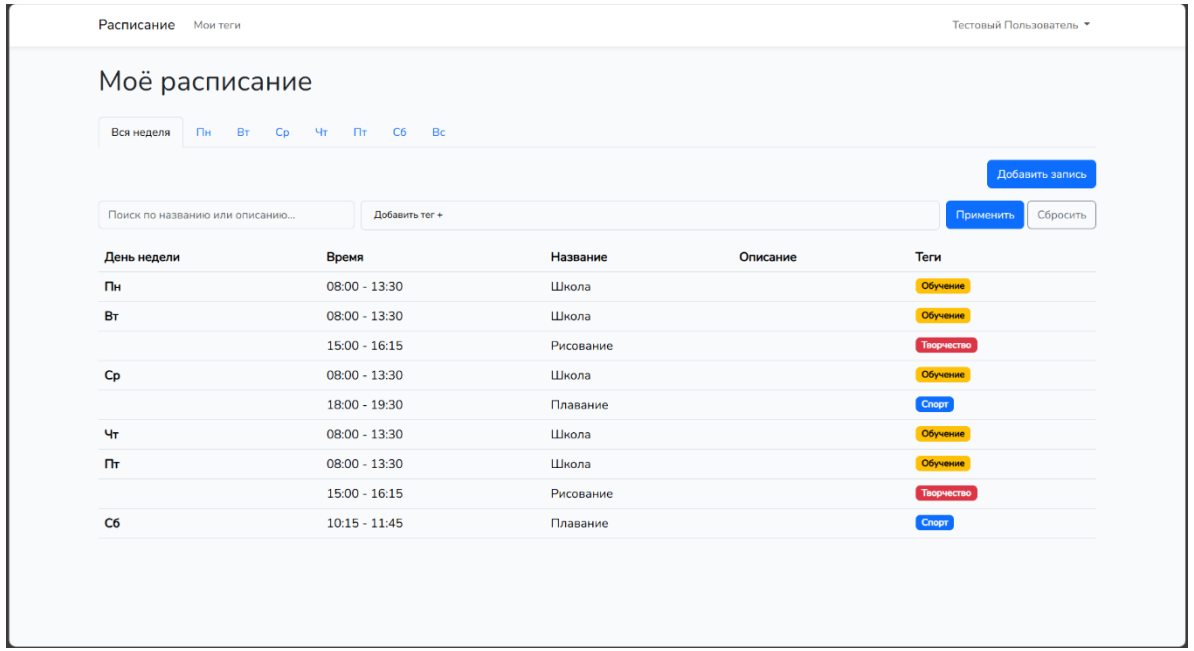

Рисунок 19 – Страница управления расписанием

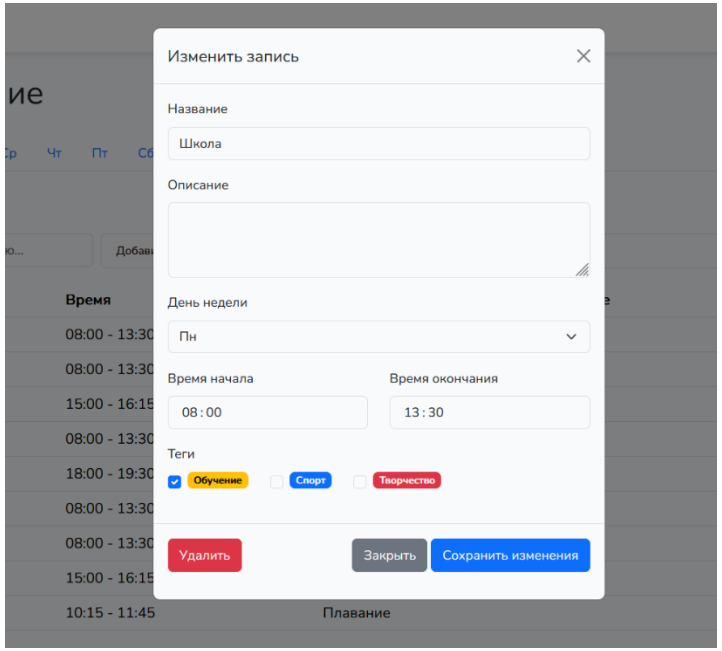

Рисунок 20 – Модальное окно изменения записи в расписании

Итоговый вид веб-приложения

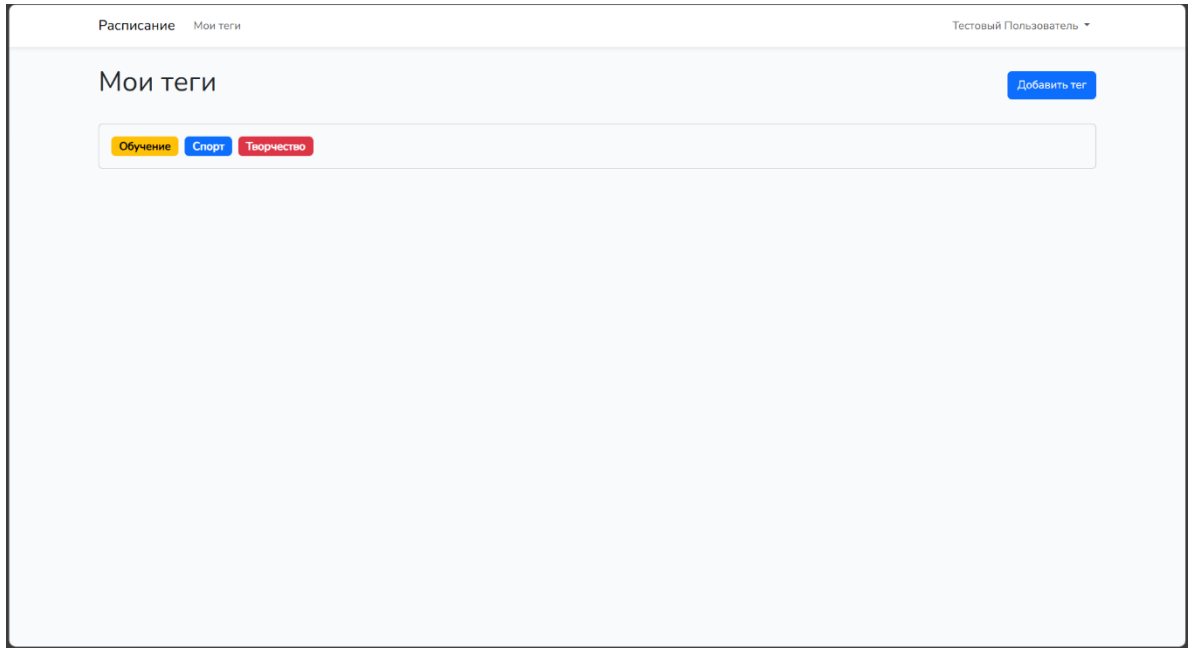

Рисунок 21 – Страница управления тегами

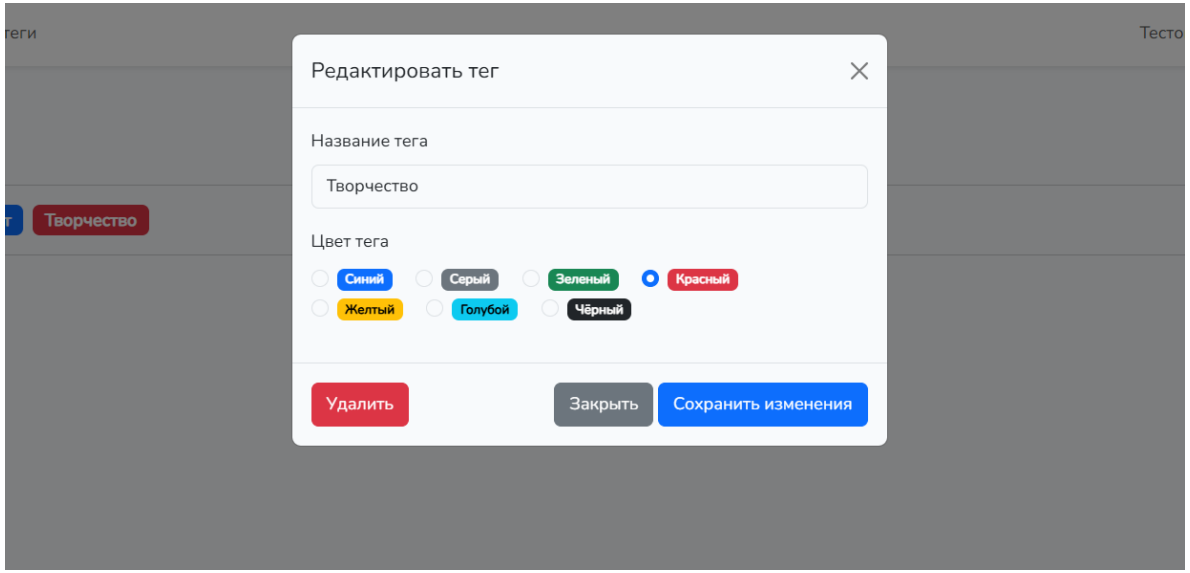

Рисунок 22 – Модальное окно редактирования тега

СПАСИБО ЗА ВНИМАНИЕ!